

DISKUSI 3
MATA KULIAH JARINGAN KOMPUTER

Kelompok 1 : Kelompok Ceria

Nama dan peran anggota :

NAMA	NIM	PERAN
Misael Arafian Fonataba	103072400017	Network Investigator
Calvin Immanuel Lado	103072400158	Application Defender
Ibrahimovich Paradise	103072400122	User Advocate
Krisna putra wicaksana	103072400079	Policy Maker

Nama Wifi : Baracafelt1

HASIL DATA LAPANGAN

Langkah 1: Buka Banyak Aplikasi Sekaligus

1. Kondisi Pengujian

Pengujian dilakukan dengan mensimulasikan beban jaringan (*stress test*), di mana kami menjalankan beberapa aplikasi secara bersamaan dalam satu perangkat (laptop) yang sama. Total tab browser yang terbuka adalah 18 tab.

2. Hasil Observasi Kinerja Aplikasi

Saat semua proses (YouTube, WhatsApp Web, dan pencarian Google) dipaksa berjalan secara bersamaan, kami mengamati anomali performa berikut:

a. YouTube

Video tidak berjalan mulus secara instan. Terjadi *buffering* selama sepersekian detik sebelum video dapat diputar dan menstabilkan resolusinya.

b. WhatsApp Web

Kinerja tetap optimal. Pesan teks dapat terkirim dengan cepat tanpa adanya *delay* yang berarti, seolah tidak terdampak oleh beban jaringan.

c. Google Search

Kinerja terasa sedikit lambat. Waktu respons (*loading*) saat memuat hasil pencarian menjadi lebih lama dari biasanya.

3. Kesimpulan

Berdasarkan pengujian, aplikasi yang paling terdampak oleh pembebanan jaringan (*multiplexing*) ini adalah Google Search (aktivitas *browsing* web). Sebagai perbandingan, ketika kami menghentikan simulasi dan mencoba membuka Google Search sendirian (tanpa memutar YouTube dan aplikasi lainnya secara bersamaan), proses *loading* pencarian kembali menjadi jauh lebih cepat dan responsif. Hal ini membuktikan adanya persaingan alokasi jaringan (perebutan *bandwidth*) antar tab dan aplikasi yang berjalan di latar belakang.

Langkah 2: Coba Kirim File Besar

1. Kondisi Pengujian Pengiriman File

Pengujian dilakukan dengan mengirimkan sebuah file berukuran besar kepada anggota kelompok lain untuk mengamati perilaku protokol transmisi data.

a. Ukuran File : 23 MB.

b. Media Pengiriman : WhatsApp Web.

2. Hasil Observasi Proses Pengiriman

Berdasarkan pengamatan visual selama pengiriman berlangsung, kami mencatat beberapa hal berikut:

a. Durasi Pengiriman

Proses pengunggahan berlangsung sangat cepat, memakan waktu kurang dari 2 detik.

b. Indikator Proses

Terdapat animasi *loading* yang sempat terlihat sesaat, menunjukkan bahwa proses pemotongan dan pengiriman data (segmentasi) sedang berjalan.

c. Status Akhir

Sistem memberikan konfirmasi visual berupa tanda centang (berhasil terkirim) tanpa memunculkan pesan *error* sama sekali.

3. Konfirmasi dan Validasi Integritas Data

Setelah file diterima, pihak penerima berhasil mengunduh dan membuka file tersebut secara normal. Seluruh isi file dapat diakses dengan lengkap tanpa ada bagian yang hilang.

Bukti Kesamaan File Dari hasil diskusi tim, kami menyimpulkan bahwa bukti teknis mutlak untuk memastikan file yang diterima *sama persis* (bit-per-bit) dengan file asli adalah dengan membandingkan nilai Hash-nya. Jika kedua file menghasilkan deret karakter *hash* yang identik, maka dipastikan tidak ada satu pun data yang rusak selama transmisi.

4. Pengamatan Tambahan: Analisis DNS TTL (Time to Live)

Selain mengirim file, kami melakukan pengecekan nilai TTL menggunakan perintah *nslookup* untuk menganalisis beban *query* aplikasi ke jaringan.

a. Data TTL WhatsApp memiliki TTL 7 detik, sedangkan YouTube memiliki TTL 461 detik.

b. Kalkulasi Frekuensi Query (Per Jam):

a. WhatsApp : $3600 \text{ detik} / 7 = 514 \text{ kali DNS query/jam}$.

b. YouTube : $600 \text{ detik} / 461 = 8 \text{ kali DNS query/jam}$.

c. Kesimpulan Analisis

WhatsApp jauh lebih sering melakukan *query* DNS dibandingkan YouTube. Namun, alasan WhatsApp tetap lancar saat jaringan *congestion* (macet) adalah karena muatan (*payload*) data per pesannya sangat kecil (umumnya $< 1 \text{ KB}$). Fakta ini membuktikan bahwa beban jaringan tidak semata-mata ditentukan

oleh tingginya frekuensi inisiasi koneksi/DNS, melainkan oleh rasio ukuran paket data yang ditransmisikan.

Langkah 3: Rasakan Perbedaan Kecepatan (Waktu Muat Halaman)

1. Kondisi Pengujian Pengujian dilakukan untuk mengukur waktu muat (*page load time*) sebuah halaman *website* menggunakan *stopwatch*, guna membandingkan kecepatan antara akses pertama (kondisi *fresh*) dan akses kedua. Website yang Diuji: <https://www.detik.com/>

2. Hasil Observasi Waktu Muat (*Loading Time*) Berdasarkan pengukuran manual beruntun dengan jeda 10 detik, kami mencatat data waktu sebagai berikut:

- a. Kunjungan Pertama (Awal) : 3,57 detik.
- b. Kunjungan Kedua (Ke-2) : 3,30 detik.
- c. Selisih Waktu Loading : 0,27 detik.
- d. Kesimpulan

Sesuai ekspektasi, proses pemuatan halaman *website* pada kunjungan kedua berjalan lebih cepat dibandingkan kunjungan pertama.

3. Analisis Teknis (Mengapa Kunjungan Kedua Lebih Cepat?)

Dari hasil diskusi tim mengenai perbedaan waktu pemuatan tersebut, kami menyimpulkan bahwa hal ini disebabkan oleh mekanisme *Caching* dan penghematan fase *Setup*:

- a. Browser Caching (Aset Statis)

Pada kunjungan pertama, *browser* harus mengunduh seluruh elemen secara utuh dari *server* (HTML, CSS, JavaScript, gambar, dll). Setelah selesai, *browser* menyimpan elemen-elemen statis tersebut di penyimpanan lokal (*Cache*). Pada kunjungan kedua, *browser* tidak perlu mengunduh ulang aset yang sama dari internet, melainkan langsung merendernya dari memori lokal.

- b. DNS Caching

Pada akses awal, sistem operasi dan *browser* harus melakukan *DNS Lookup* (bertanya ke *DNS Server*) untuk menerjemahkan nama *domain* menjadi *IP Address*. Pada akses kedua, hasil *query* IP tersebut sudah tersimpan di *DNS Cache* lokal, sehingga memangkas waktu tunggu *Round Trip Time* (RTT) untuk pencarian alamat.

c. TCP Connection Reuse (Keep-Alive)

Karena jarak antar akses hanya 10 detik, sangat besar kemungkinan koneksi TCP yang dibuat pada kunjungan pertama belum diputus (masih berstatus *ESTABLISHED* atau *keep-alive*). Sehingga, kunjungan kedua tidak perlu lagi membuang waktu untuk melakukan TCP 3-way *Handshake* dan TLS *Setup* dari nol.

Langkah 4: Amati Saat Jaringan 'Macet'

1. Kondisi Pengujian

Pengujian dilakukan dengan menyimulasikan pembebanan jaringan (*stress test*) secara paralel, di mana seluruh anggota kelompok membuka YouTube di masing-masing *smartphone* (total 4 perangkat) dan memutar video secara bersamaan.

- a. Durasi Pengujian : 2 menit.
- b. Aplikasi Tambahan : WhatsApp (untuk pengujian dampak kelambatan pada aplikasi lain).

2. Hasil Observasi Kinerja Aplikasi

Selama simulasi pembebanan jaringan berlangsung, kami mencatat anomali dan performa sebagai berikut:

a. Stabilitas Pemutaran Video

Tidak terjadi penurunan kualitas secara otomatis (dinamis) di tengah-tengah pemutaran video, dan tidak ada perangkat yang mengalami indikasi *buffering* selama durasi 2 menit.

b. Perbedaan Resolusi Inisiasi

Walaupun stabil, sistem memberikan alokasi resolusi awal yang berbeda. Tiga 3 perangkat berhasil menginisiasi pemutaran di kualitas tinggi (720p), sementara satu 1 perangkat menginisiasi pemutaran di kualitas yang lebih rendah (sekitar 480p/420p). Resolusi ini bertahan statis selama pengujian.

c. Kinerja WhatsApp: Saat dilakukan percobaan pengiriman pesan teks melalui WhatsApp di tengah berjalannya keempat video resolusi tinggi tersebut, pesan tetap terkirim seketika tanpa mengalami *delay* sedikit pun.

3. Analisis Teknis

Dari hasil observasi yang unik tersebut, kami menyimpulkan dua teknis:

a. Adaptive Bitrate Streaming pada Fase Setup

Aplikasi *streaming* seperti YouTube selalu mengukur estimasi kecepatan jaringan pada awal koneksi. Karena 4 perangkat meminta data secara serentak, *router* membagi antrean (*bandwidth*) secara kompetitif. Satu perangkat mendeteksi bahwa jatah kecepatan yang ia dapatkan saat itu lebih kecil, sehingga algoritma YouTube mengambil tindakan preventif dengan memberikan resolusi awal yang lebih rendah demi menghindari *buffering* sebelum video sempat diputar.

b. Sisa Kapasitas dan Karakteristik Payload

Fakta bahwa video tidak macet dan pesan WhatsApp tetap terkirim lancar membuktikan bahwa kapasitas total *bandwidth* WiFi yang kami gunakan sebenarnya masih cukup besar untuk melayani lalu lintas 4 perangkat. Pesan WhatsApp tidak terhambat karena ukuran datanya (*payload*) sangat kecil (di bawah 1 KB), sehingga dapat dengan mudah "menyelip" dan langsung diproses di sela-sela paket data video yang besar.

HASIL PERTANYAAN DISKUSI

Soal 1 :

Berdasarkan diskusi kelompok, kami menyimpulkan bahwa fenomena perlambatan yang dialami Rafif saat mengupload file merupakan dampak langsung dari persaingan bandwidth (multiplexing) antar aplikasi yang berjalan secara bersamaan dalam satu perangkat. Meskipun secara fisik hanya menggunakan satu jaringan dan satu IP address, setiap aplikasi membuka port lokal yang berbeda dan berkompetisi untuk mendapatkan alokasi bandwidth yang sama. Aplikasi dengan kebutuhan bandwidth besar seperti sistem upload akan paling terdampak, sementara aplikasi dengan payload kecil seperti WhatsApp tetap berjalan lancar karena tidak membutuhkan porsi bandwidth yang signifikan.

[Policy Maker]: Perlu diadakan aturan dalam membatasi jumlah koneksi untuk batas tertentu agar dapat menjalankan dan menggunakan internet dengan *speed* atau kecepatan yang lancar.

[Application Defender]: Dari sisi aplikasi, sistem upload harus bersaing dengan berbagai proses lain yang berjalan secara bersamaan dalam satu perangkat, seperti Zoom, browser, dan layanan sinkronisasi cloud. Semua aplikasi tersebut menggunakan bandwidth yang sama, sehingga aplikasi upload yang membutuhkan bandwidth besar dan stabil cenderung mendapatkan porsi yang lebih kecil ketika jaringan sedang padat

Selain itu, sistem upload yang masih menggunakan mekanisme pengiriman file secara langsung (single stream) menjadi kurang adaptif terhadap kondisi jaringan yang dinamis. Ketika bandwidth terbatas dan terjadi kompetisi antar aplikasi, performa upload akan menurun secara signifikan.

Oleh karena itu, aplikasi seharusnya dirancang lebih adaptif dengan menerapkan mekanisme seperti chunked upload agar file dikirim dalam bagian-bagian kecil, serta resume upload untuk melanjutkan proses jika terjadi gangguan. Dengan pendekatan ini, aplikasi dapat tetap berjalan lebih stabil meskipun harus berbagi bandwidth dengan proses lain.

[User Advocate]: Melihat kerumitan *multiplexing* ini, saya menyadari betapa buruknya pengalaman Rafif sebagai pengguna awam. Pengguna tidak paham soal *port* atau persaingan *bandwidth* di latar belakang. Rafif hanya tahu bahwa sistem upload kampus "macet". Seharusnya, aplikasi memiliki indikator visual yang memperingatkan pengguna jika mendeteksi adanya persaingan jaringan yang tinggi di perangkat mereka.

[Network Investigator]: Dari data koneksi yang saya amati, terlihat bahwa laptop Rafif membuka banyak koneksi jaringan secara bersamaan. Meskipun hanya menggunakan satu jaringan dan satu IP *address*, setiap aplikasi menggunakan *port* lokal yang berbeda-beda.

Soal 2

Berdasarkan diskusi kelompok, kami menyimpulkan bahwa perbedaan performa antar aplikasi saat jaringan padat tidak semata-mata ditentukan oleh frekuensi DNS query, melainkan oleh ukuran payload data yang ditransmisikan. WhatsApp yang memiliki TTL sangat rendah (7 detik) memang jauh lebih sering melakukan DNS query dibandingkan YouTube (TTL 461 detik), namun tetap lancar karena setiap transaksinya hanya membawa data berukuran sangat kecil. Sebaliknya, upload file 80 MB harus terus-menerus bersaing memperebutkan bandwidth dengan Zoom dan YouTube yang mengalirkan data dalam jumlah besar secara berkelanjutan. Selain itu, perbedaan protokol transport (UDP vs TCP) juga berpengaruh signifikan terhadap bagaimana masing-masing aplikasi merespons kondisi jaringan yang congested.

Dari hasil nslookup, kami menemukan perbedaan TTL yang signifikan antar domain. YouTube memiliki TTL 461 detik, sehingga dalam satu jam hanya melakukan DNS query sekitar 8 kali. Sebaliknya, whatsapp.com memiliki TTL hanya 7 detik, artinya dalam satu jam WhatsApp melakukan DNS query hingga 514 kali. Namun meskipun WhatsApp lebih sering melakukan DNS query, aplikasi ini tetap lancar karena ukuran data per transaksinya sangat kecil satu pesan teks hanya beberapa ratus byte, jauh berbeda dengan stream video YouTube yang terus-menerus mengalirkan data besar.

Dari sisi protokol transport, Zoom dan game online menggunakan UDP karena memprioritaskan kecepatan dan toleran terhadap packet loss. Upload file dan WhatsApp chat menggunakan TCP yang reliable. Saat jaringan padat, aplikasi berbasis UDP seperti Zoom langsung terdampak karena tidak ada mekanisme retransmisi packet yang hilang tidak dikirim ulang. Sementara TCP melakukan retransmisi tapi ini justru memperlambat upload Rafif karena harus menunggu konfirmasi tiap segment. WhatsApp tetap lancar karena meski pakai TCP, ukuran datanya sangat kecil sehingga selesai dalam hitungan milidetik dan tidak bersaing berat dengan aplikasi lain.

[Network Investigator] menambahkan: TTL whatsapp.com yang hanya 7 detik menunjukkan WhatsApp menggunakan TTL rendah kemungkinan untuk *load balancing* server WhatsApp bisa berubah-ubah secara dinamis. Dari sisi jaringan, ini berarti lebih banyak DNS *query*, tapi karena ukuran *payload* TCP-nya kecil, dampaknya ke *bandwidth*

sangat minimal dibanding YouTube yang TTL-nya 461 detik tapi data per koneksinya jauh lebih besar.

[Application Defender] menambahkan: Sistem upload kampus tidak memiliki mekanisme prioritas *traffic*. Seharusnya ada QoS yang mendahulukan koneksi ke website tersebut apalagi TTL-nya hanya 15-16 detik, artinya koneksi sering diperbarui dan rentan terganggu saat jaringan padat.

[User Advocate] menambahkan: Yang dirasakan Rafif masuk akal WhatsApp tetap bisa dipakai karena pesannya kecil dan cepat selesai, sementara upload file 80 MB harus bersaing terus-menerus dengan Zoom dan YouTube untuk mendapat *bandwidth*. Pengguna tidak sadar soal protokol, mereka hanya merasakan upload-nya *stuck* tapi *chat* lancar.

Soal 3

Berdasarkan diskusi kelompok, kami menyimpulkan bahwa hasil pengujian ping ke LMS kampus menunjukkan kondisi jaringan yang tidak stabil secara signifikan. Meskipun tidak terjadi packet loss sama sekali (0%), nilai jitter yang sangat tinggi antara 18 ms hingga 403 ms mengindikasikan adanya bottleneck parah pada salah satu dari 6 router yang dilalui paket data. Kondisi ini membuktikan bahwa jaminan reliable delivery yang diberikan TCP pada layer transport tidak cukup untuk memastikan integritas file secara menyeluruh di sisi aplikasi. Oleh karena itu, diperlukan mekanisme verifikasi tambahan di level aplikasi, seperti checksum atau hash, untuk memastikan file yang diterima server benar-benar identik dengan file asli yang dikirim oleh mahasiswa.

Berdasarkan pengujian *ping* ke LMS kampus, jaringan terindikasi sangat tidak stabil dengan RTT yang berfluktuasi ekstrem antara 18 ms hingga 403 ms (*jitter* tinggi). Meski demikian, transmisi tingkat dasar tetap andal dengan catatan 0% *packet loss*. Selain itu, nilai balasan TTL sebesar 58 menunjukkan bahwa paket data melompat melewati 6 *router* (6 *hop*), yang dihitung dari asumsi nilai TTL *default* awal (64) dikurangi TTL balasan (58).

[Network Investigator]: Sebagai pihak jaringan, yang menjadi sorotan saya bukan *packet loss*-nya yang 0%, melainkan *jitter* yang luar biasa tinggi (selisih hingga 385ms antar paket). Lonjakan waktu hingga 403ms ini berarti antrean di salah satu dari 6 *router* yang dilewati paket sedang sangat *bottleneck*.

[Application Defender]: Meskipun TCP menyediakan mekanisme reliable delivery seperti error checking, retransmission, dan sequencing, jaminan tersebut hanya berlaku pada level transport, yaitu memastikan data sampai ke tujuan tanpa kesalahan bit selama transmisi.

TCP tidak menjamin bahwa file yang diterima di sisi aplikasi benar-benar utuh, lengkap, dan sesuai dengan file asli yang dikirim. Oleh karena itu, sistem upload kampus seharusnya menambahkan lapisan verifikasi di level aplikasi, seperti checksum atau hash (misalnya MD5 atau SHA) untuk memastikan integritas file setelah proses upload selesai. Sistem dapat membandingkan nilai hash file di sisi client dan server untuk memastikan tidak ada perubahan atau kerusakan data.

Selain itu, sistem juga dapat menerapkan validasi ukuran file (file size check), serta mekanisme chunked upload dengan verifikasi per bagian, sehingga jika terjadi kegagalan atau inkonsistensi pada sebagian data, hanya bagian tersebut yang perlu dikirim ulang. Dengan demikian, mengandalkan TCP saja tidak cukup, karena integritas file secara keseluruhan harus diverifikasi di level aplikasi, bukan hanya pada proses pengiriman data di jaringan.

[User Advocate]: Mengingat ada risiko *corrupt* secara teknis yang di luar kendali mahasiswa, aplikasi kampus wajib menyediakan fitur "Preview File" atau "Download Ulang" setelah proses upload selesai. Mahasiswa harus bisa mengecek sendiri apakah file yang terkirim ke server hancur atau tidak sebelum *deadline* ditutup.

[Policy Maker]: Berdasarkan temuan bahwa keandalan sistem dapat gagal dan file bisa *corrupt* murni karena kondisi teknis *bottleneck* (di luar kendali mahasiswa), saya menetapkan kebijakan Mahasiswa yang bisa membuktikan bahwa file mereka *corrupt* akibat jam sibuk sistem harus diberikan kompensasi perpanjangan waktu pengumpulan.

Soal 4

1. Step 1 Jumlah Segment

Ukuran file	: $80 \text{ MB} = 80 \times 1.048.576 \times 8 = 671.088.640 \text{ bit}$
Segment size	: $1460 \text{ byte} = 1460 \times 8 = 11.680 \text{ bit}$
Jumlah segment	: $671.088.640 / 11.680 = 57.456 \text{ segment}$

2. Step 2 Waktu Upload Stop-and-Wait

Rumus:

Waktu per segment = Waktu transmisi + RTT

Waktu transmisi = Segment size ÷ Bandwidth upload

$$= 11.680 \text{ bit} \div 47.390.000 \text{ bit/s}$$

$$= 0,000246 \text{ detik} = 0,246 \text{ ms}$$

$$\text{RTT ke LMS} = 113 \text{ ms}$$

$$\text{Waktu per segment} = 0,246 \text{ ms} + 113 \text{ ms} = 113,246 \text{ ms}$$

$$\text{Total waktu stop-and-wait} = 57.456 \times 113,246 \text{ ms}$$

$$= 6.507.788 \text{ ms}$$

$$= \pm 6.508 \text{ detik}$$

$$= \pm 108 \text{ menit!}$$

3. Step 3 Utilization Stop-and-Wait

$$\text{Utilization} = \text{Waktu transmisi} \div (\text{Waktu transmisi} + \text{RTT})$$

$$= 0,246 \div (0,246 + 113)$$

$$= 0,246 \div 113,246$$

$$= 0,00217$$

$$= 0,22\% \leftarrow \text{sangat buruk!}$$

4. Step 4 Utilization Pipelining Window 10 Segment

$$\text{Utilization} = (10 \times 0,246) \div 113,246$$

$$= 2,46 \div 113,246$$

$$= 2,17\%$$

Masih kecil tapi 10x lebih baik dari stop-and-wait.

5. Step 5 Window Size untuk Utilization 50%

$$0,50 = (W \times 0,246) \div 113,246$$

$$W = (0,50 \times 113,246) \div 0,246$$

$$W = 56,623 \div 0,246$$

$$W = \pm 230 \text{ segment}$$

Butuh 230 segment paralel agar utilization mencapai 50%.

Dengan upload 47,39 Mbps dan RTT 113ms ke lms.telkomuniversity.ac.id, upload file 80 MB secara stop-and-wait membutuhkan waktu teoritis 108 menit dengan utilization hanya 0,22%. Ini membuktikan masalah malam itu bukan semata-mata karena bandwidth kurang upload speed 47,39 Mbps sebenarnya cukup besar. Masalah utamanya adalah RTT yang tinggi dan tidak stabilnya jaringan, sehingga stop-and-wait menjadi sangat tidak efisien.

Berdasarkan diskusi kelompok, kami menyimpulkan bahwa masalah yang dialami Rafif saat mengupload file 80 MB bukanlah semata-mata karena kekurangan bandwidth. Dengan kecepatan upload 47,39 Mbps, secara teoritis bandwidth yang tersedia sudah lebih dari cukup. Permasalahan utama justru terletak pada tingginya nilai RTT (113 ms)

dikombinasikan dengan penggunaan protokol stop-and-wait, yang menghasilkan utilisasi bandwidth hanya sebesar 0,22%. Kondisi ini membuktikan bahwa menambah kapasitas bandwidth tanpa memperbaiki arsitektur protokol dan mereduksi latensi tidak akan menyelesaikan masalah secara fundamental.

[Policy Maker]: Dari perhitungan ini, menambah *bandwidth* saja tidak cukup. Dengan RTT 113ms, bahkan *bandwidth* besar pun akan terbuang sia-sia kalau protokolnya *stop-and-wait*. Solusi lebih fundamental adalah memastikan sistem upload kampus menggunakan TCP *window sizing* yang optimal dan mempertimbangkan penerapan QUIC/HTTP3 untuk mengurangi *overhead handshake*.

[Network Investigator]: *Utilization* 0,22% menunjukkan *bandwidth* yang terpakai sangat sedikit dari yang tersedia. Dengan *pipelining window* 230 *segment* baru bisa mencapai 50% *utilization* ini menunjukkan betapa besarnya pengaruh RTT terhadap efisiensi transfer data. Perhitungan ini membuktikan bahwa *bottleneck* sesungguhnya ada pada latensi (RTT), bukan pada kapasitas *throughput* saluran. Kecepatan upload 47 Mbps menjadi sama sekali tidak berguna jika kita harus menunggu *acknowledgment* selama 113 ms untuk setiap pengiriman data sekecil 1460 *byte*.

[Application Defender]: Melihat hitungan ini, aplikasi kampus harus dirancang untuk mendukung *pipelining* agresif atau beralih dari protokol TCP standar. Jika aplikasi atau server membatasi *window size* penerimaan terlalu kecil, kita tidak akan pernah bisa mengirim 230 *segment* secara paralel untuk mencapai utilisasi 50% tadi.

[User Advocate]: Menunggu 108 menit untuk mengunggah satu file tugas 80 MB adalah hal yang sangat tidak masuk akal dan merugikan mahasiswa secara waktu. Apalagi jika di menit ke-100 terjadi *error* dan sistem tidak bisa *resume upload*, mahasiswa bisa gagal mengumpulkan tugas tepat waktu.

[Policy Maker]: Berdasarkan data utilisasi matematis ini, saya dapat menyimpulkan bahwa solusi dari masalah jaringan malam itu tidak akan selesai hanya dengan menambah *bandwidth* berlangganan kampus. Ada masalah fundamental pada arsitektur jaringan yang perlu diubah. Kita perlu menginvestasikan dana untuk mendekatkan server fisik LMS ke area asrama guna memangkas RTT, atau meminta tim IT merombak sistem web agar menggunakan protokol modern (seperti QUIC) yang dirancang untuk mengatasi latensi tinggi.

Soal 5

AIMD atau *Additive Increase Multiplicative Decrease* merupakan mekanisme algoritma utama yang digunakan oleh protokol TCP (*Transmission Control Protocol*) untuk mengendalikan kemacetan jaringan (*congestion control*). Secara mendasar, AIMD bertugas mengatur seberapa banyak volume data yang diizinkan untuk dikirimkan sekaligus dalam satu waktu. Cara kerja algoritma ini dapat diibaratkan seperti perilaku seorang pengemudi kendaraan di jalan tol yang menyesuaikan kecepatannya dengan kondisi lalu lintas.

Satu koneksi download sudah membuat latency naik dari 70ms ke 696ms. Dengan 20 koneksi paralel Dika, buffer router akan penuh jauh lebih cepat dan lebih konsisten setiap kali ada multiplicative decrease di salah satu koneksi Dika, 19 koneksi lainnya masih terus mengirim data dan mengisi buffer. Akibatnya latency mahasiswa lain akan terus tinggi tanpa jeda

Berdasarkan diskusi kelompok, kami menyimpulkan bahwa tindakan Dika yang membuka 20 koneksi paralel secara otomatis menggunakan script telah menciptakan efek domino yang merugikan seluruh pengguna jaringan di asrama. Fenomena ini dikenal sebagai Bufferbloat, di mana buffer router menjadi penuh secara konsisten akibat banjir paket dari satu host, sehingga latensi mahasiswa lain melonjak drastis dari 70 ms menjadi 696 ms. Fakta ini mengungkap kerentanan mendasar pada arsitektur jaringan kampus yang masih mengandalkan antrean FIFO standar, tanpa mekanisme pembatasan koneksi per pengguna maupun sistem fair queuing berbasis IP.

[Network Investigator]: Secara teknis jaringan, fenomena ini disebut sebagai *Bufferbloat*. Mengingat kami tidak memiliki peran *Security Analyst*, saya bertugas menganalisis polanya: trafik otomatis dari *script* seperti milik Dika sangat mudah diidentifikasi di level *router*. Indikator utamanya adalah adanya lonjakan pengiriman paket SYN (TCP *Handshake*) secara masif ke alamat IP dan *port* tujuan yang sama (LMS kampus) dari satu IP *host* asal dalam rentang waktu sepersekian detik.

[Application Defender]: Dari sisi aplikasi web kampus, ini adalah kelemahan arsitektur. Kami tidak boleh pasrah pada algoritma *fairness* TCP. Sistem aplikasi harus menerapkan mekanisme *Rate Limiting* berdasarkan alamat IP, bukan hanya per sesi koneksi. Selain itu, implementasi CAPTCHA atau *token-based authentication* bisa mencegah *script* otomatis membuka banyak sesi *refresh* secara liar.

[User Advocate]: Tindakan egois atau ketidaksengajaan satu pengguna ternyata bisa menciptakan efek domino (latensi 696ms) yang melumpuhkan aktivitas akademik satu

asrama. Ini menunjukkan betapa rentannya keadilan jaringan kampus. Mahasiswa yang jujur menggunakan 1 *tab* terhukum oleh desain protokol, sementara yang menggunakan *script* pihak ketiga justru diuntungkan.

[Policy Maker]: Untuk mengatasi masalah ketimpangan ini, kami di manajemen kampus harus membuat keputusan strategis: jaringan kampus tidak boleh lagi menggunakan antrean standar (FIFO) di *router* asrama. Kami perlu mengimplementasikan *Fair Queuing* berbasis IP (seperti FQ-CoDel atau SFQ). Dengan kebijakan antrean ini, sebesar apa pun Dika membuka puluhan koneksi TCP, total *bandwidth* yang dia dapatkan akan tetap disamakan dengan laptop mahasiswa lain yang hanya membuka satu koneksi.

Soal 6

Berdasarkan diskusi kelompok, kami menyimpulkan bahwa pemborosan sumber daya yang terjadi pada saat surge traffic malam pengumpulan tugas bersumber dari ketidakcocokan protokol yang digunakan. Fakta bahwa 76,5% dari total RTT terbuang hanya untuk proses inisiasi koneksi (handshake) membuktikan bahwa HTTP/1.1 atau HTTP/2 di atas TCP konvensional tidak dirancang untuk menangani lonjakan koneksi simultan dalam jumlah besar. Migrasi ke HTTP/3 berbasis QUIC merupakan solusi paling hemat biaya dan berdampak paling besar yang dapat segera diimplementasikan, tanpa perlu investasi perangkat keras baru.

[Network Investigator]: Sebagai analis teknis, melihat 76,5% RTT terbuang untuk proses inisiasi adalah mimpi buruk efisiensi. Ini membuktikan bahwa protokol HTTP/1.1 atau HTTP/2 yang berjalan di atas TCP konvensional sangat tidak cocok untuk situasi *surge traffic* (lonjakan lalu lintas mendadak) seperti malam pengumpulan tugas.

[Application Defender]: Dari sisi pengembang, ini menyadarkan kami bahwa aplikasi web kampus (LMS) tidak boleh tertinggal zaman. Kami harus segera memperbarui konfigurasi web server (seperti Nginx atau Apache) agar mendukung HTTP/3 (QUIC). Dukungan ini akan sangat melegakan beban CPU server saat menangani ribuan mahasiswa yang *login* bersamaan.

[User Advocate]: Dari kacamata mahasiswa awam, mereka tentu merasakan perbedaan secara psikologis antara HTTP/1.1 dan HTTP/3. Pengguna tidak peduli pada *handshake*; mereka hanya melihat layar putih (*blank screen*) selama proses inisiasi 183 ms itu. Dengan HTTP/3, *loading* terasa instan tanpa layar putih yang lama. Cara menjelaskan komplain internet kampus lambat ke mahasiswa: "Sistem lama ibarat kamu harus lapor KTP ke 3 pos satpam berbeda sebelum boleh masuk bawa barang. Besok kita *upgrade* sistem baru, di

mana kamu cukup tunjuk wajah sekali di gerbang dan langsung bisa lempar file tugas ke dalam."

[Policy Maker]: Menyaksikan data kalkulasi pemborosan sumber daya server selama 55 detik ini, keputusan manajemen menjadi sangat jelas. Tidak perlu membeli perangkat keras baru. Kebijakan jangka pendek yang paling hemat biaya namun berdampak masif adalah mewajibkan divisi IT melakukan migrasi protokol server asrama ke HTTP/3 secara menyeluruh pada semester depan.

Soal 7

Berdasarkan diskusi kelompok, kami menyimpulkan bahwa permasalahan yang dialami mahasiswa saat mengumpulkan tugas merupakan hasil akumulasi dari beberapa kelemahan yang terjadi secara bersamaan, mulai dari *queuing delay* parah di sisi server yang tercermin dari TCP handshake 92,15 ms jauh melampaui RTT idle 4 ms, hingga absennya mekanisme verifikasi integritas file di level aplikasi. Bandwidth upload 47,39 Mbps yang tersedia sebenarnya secara teoritis sudah mencukupi, namun menjadi tidak efektif akibat distribusi yang tidak adil karena koneksi paralel berlebihan, overhead protokol yang tinggi, serta tidak adanya sistem prioritas traffic. Solusi yang diperlukan bersifat menyeluruh, mencakup pembenahan pada layer jaringan, layer aplikasi, maupun kebijakan penggunaan sistem.

[Network Investigator]: Temuan paling kritis adalah TCP *handshake* 92.15 ms yang jauh melampaui RTT *idle* 4 ms. Ini menunjukkan server kampus mengalami *queuing delay* parah saat diakses ramai. Prioritas teknis pertama adalah mengurangi jumlah koneksi yang perlu *di-handshake*, bukan sekadar menambah *bandwidth*.

[Application Defender]: Sistem upload kampus harus segera menambahkan verifikasi *hash* di sisi server bandingkan *checksum* file yang diterima dengan *checksum* yang dikirim *client*. TCP menjamin data sampai urut dan lengkap, tapi tidak menjamin file tidak *corrupt* akibat *bug* aplikasi, *disk error*, atau *memory issue* di server. Ini bukan tugas TCP, ini tugas aplikasi.

[User Advocate]: Dari sisi mahasiswa, masalah terburuk bukan lambatnya upload tapi tidak adanya *feedback*. Tiga file *corrupt* terupload tanpa peringatan apapun. Mahasiswa merasa sudah mengumpulkan tugas, padahal tidak. Solusi termudah dan tercepat yang bisa langsung dirasakan pengguna adalah notifikasi konfirmasi upload berhasil beserta *hash* file, sehingga mahasiswa bisa verifikasi sendiri.

[Policy Maker] menambahkan: Berdasarkan data, solusinya bukan cukup dengan menambah *bandwidth* saja. *Bandwidth* upload 47.39 Mbps secara teoritis cukup,

masalahnya adalah distribusi yang tidak adil akibat koneksi paralel berlebihan dan *overhead* protokol yang tinggi. Keputusan yang perlu diambil tetapkan batas maksimal koneksi per mahasiswa, wajibkan sistem upload menggunakan verifikasi integritas file, dan buat jadwal upload bertahap agar tidak menumpuk di malam hari.

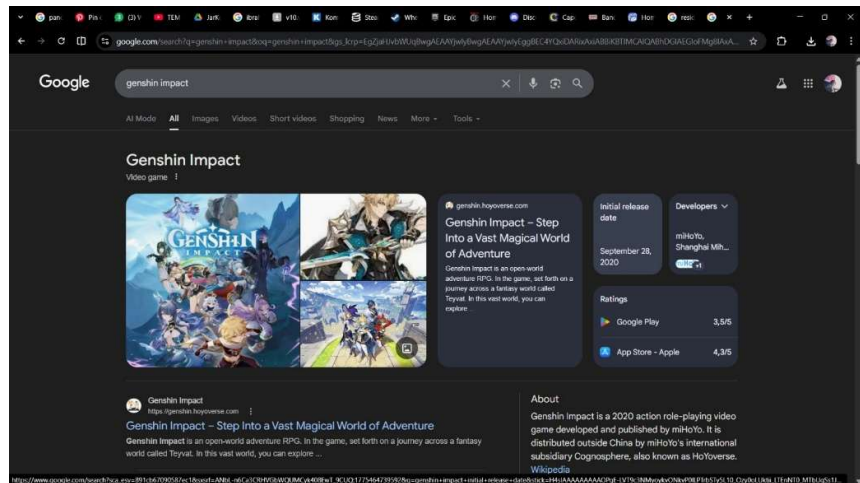
Lampiran 1

Output netstat dan Capture Browser

```
PS C:\Users\Lenovo> netstat -anp TCP
Active Connections

    Proto Local Address           Foreign Address         State
    TCP    0.0.0.0:135             0.0.0.0:0               LISTENING
    TCP    0.0.0.0:445             0.0.0.0:0               LISTENING
    TCP    0.0.0.0:2869            0.0.0.0:0               LISTENING
    TCP    0.0.0.0:5040            0.0.0.0:0               LISTENING
    TCP    0.0.0.0:7250            0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49664           0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49665           0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49666           0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49667           0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49668           0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49670           0.0.0.0:0               LISTENING
    TCP    10.218.14.82:139        0.0.0.0:0               LISTENING
    TCP    10.218.14.82:49438      4.213.25.241:443        ESTABLISHED
    TCP    10.218.14.82:50326      142.251.10.94:443       TIME_WAIT
    TCP    10.218.14.82:51032      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:51181      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:51281      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:52468      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:52931      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:53016      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:53080      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:53116      142.251.10.100:443      ESTABLISHED
    TCP    10.218.14.82:53699      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:53823      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:53829      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:53890      4.213.25.241:443        ESTABLISHED
    TCP    10.218.14.82:53942      64.233.170.84:443       ESTABLISHED
    TCP    10.218.14.82:54155      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:54768      172.188.155.25:443      ESTABLISHED
    TCP    10.218.14.82:55323      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:55572      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:55625      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:56386      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:56443      57.144.153.32:443       TIME_WAIT
    TCP    10.218.14.82:56752      52.123.170.77:443       ESTABLISHED
    TCP    10.218.14.82:56818      52.123.164.123:443      ESTABLISHED
    TCP    10.218.14.82:56907      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:57295      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:57444      23.32.244.8:443         ESTABLISHED
    TCP    10.218.14.82:57640      52.104.129.39:443       TIME_WAIT
    TCP    10.218.14.82:58481      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:58903      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:58980      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:59056      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:59625      40.99.10.18:443         TIME_WAIT
    TCP    10.218.14.82:60063      10.217.7.77:53          TIME_WAIT
    TCP    10.218.14.82:60434      10.217.7.77:53          TIME_WAIT
```

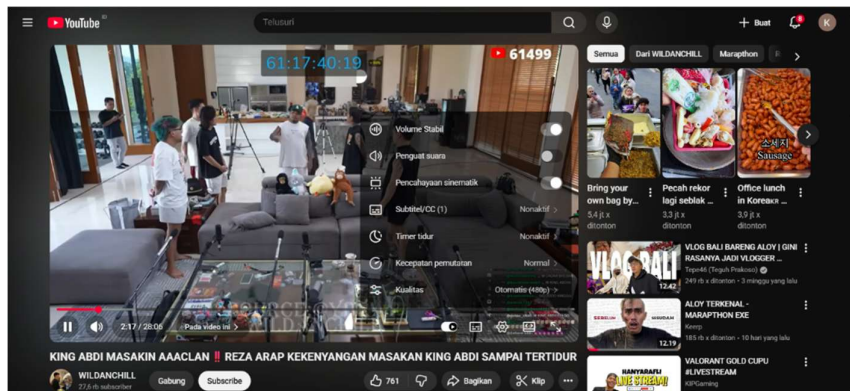
Gambar 1 Output netstat



Gambar 2 capture tab browser

Lampiran 2

Kualitas Video Youtube



Lampiran 3

Hasil ping

```
PS C:\Users\Lenovo> ping lms.telkomuniversity.ac.id -n 20

Pinging proxy-fallback.celoe.in [172.67.153.77] with 32 bytes of data:
Reply from 172.67.153.77: bytes=32 time=18ms TTL=58
Reply from 172.67.153.77: bytes=32 time=33ms TTL=58
Reply from 172.67.153.77: bytes=32 time=185ms TTL=58
Reply from 172.67.153.77: bytes=32 time=185ms TTL=58
Reply from 172.67.153.77: bytes=32 time=41ms TTL=58
Reply from 172.67.153.77: bytes=32 time=36ms TTL=58
Reply from 172.67.153.77: bytes=32 time=50ms TTL=58
Reply from 172.67.153.77: bytes=32 time=36ms TTL=58
Reply from 172.67.153.77: bytes=32 time=301ms TTL=58
Reply from 172.67.153.77: bytes=32 time=40ms TTL=58
Reply from 172.67.153.77: bytes=32 time=57ms TTL=58
Reply from 172.67.153.77: bytes=32 time=28ms TTL=58
Reply from 172.67.153.77: bytes=32 time=43ms TTL=58
Reply from 172.67.153.77: bytes=32 time=56ms TTL=58
Reply from 172.67.153.77: bytes=32 time=277ms TTL=58
Reply from 172.67.153.77: bytes=32 time=403ms TTL=58
Reply from 172.67.153.77: bytes=32 time=50ms TTL=58
Reply from 172.67.153.77: bytes=32 time=22ms TTL=58
Reply from 172.67.153.77: bytes=32 time=328ms TTL=58
Reply from 172.67.153.77: bytes=32 time=80ms TTL=58

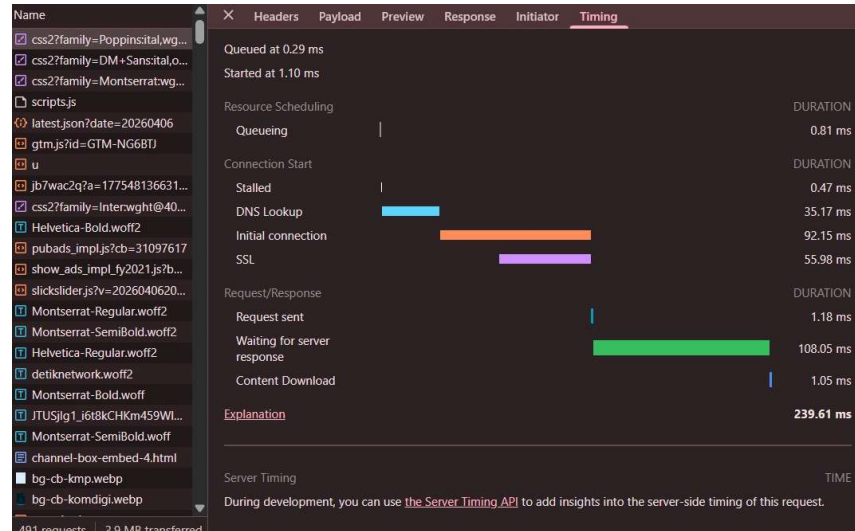
Ping statistics for 172.67.153.77:
    Packets: Sent = 20, Received = 20, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 18ms, Maximum = 403ms, Average = 113ms
```

Gambar 3 Output ping

Lampiran 4

Output devtools timing

1. Devtools timing



The screenshot shows the 'Network' tab in the Chrome DevTools Network panel. The top bar includes filters for 'Fetch/XHR', 'Doc', 'CSS', 'JS', 'Font', 'Img', 'Media', 'Manifest', 'Socket', 'Wasm', and 'Other'. The main panel displays a list of resources with columns for Name, Status, Type, Initiator, Size, and Time. The status is '200' for all resources. The size and time columns are in kB and ms respectively. The bottom bar shows '490 requests' and '3.9 MB transferred'.

Name	Status	Type	Initiator	Size	Time
css2?family=Poppinsital,wght@0,100;0,200;0,300;0,400;0,500;0,600;0,700;0,800;0,900&display=swap	200	stylesheet	style.css:1	1.4 kB	239 ms
css2?family=DM+Sansital,opsz,wght@0,9,40,100,1...&display=swap	200	stylesheet	style.css:1	0.5 kB	239 ms
css2?family=Montserratwght@100,900&display=swap	200	stylesheet	style.css:1	0.7 kB	243 ms
scripts.js	200	xhr	ebx.js:1	3.0 kB	58 ms
latest.json?date=20260406	200	fetch	11530.js:24	1.2 kB	305 ms
gtm.js?id=GTM-NG68TJ	200	script	(index):71	169 kB	1.34 s
u	200	script	(index):581	0.3 kB	454 ms
jb7wac2q?a=1775481366311&o=oxk51aznec	200	script	(index):581	5.7 kB	454 ms
css2?family=Interwght@400;600&display=swap	200	stylesheet	framebar.css?v=202604062075	0.8 kB	55 ms
Helvetica-Bold.woff2	200	font	style.css?v=202604062075	109 kB	220 ms
pubads_impl.js?cb=31097617	200	script	gpt.js:11	187 kB	407 ms
show_ads_impl_fy2021.js?bust=31097613	200	script	adsbygoogle.js?client=ca-pu	181 kB	415 ms
slickslider.js?v=202604062075	200	script	(index):612	10.9 kB	165 ms
Montserrat-Regular.woff2	200	font	style.css?v=202604062075	35.2 kB	101 ms
Montserrat-SemiBold.woff2	200	font	style.css?v=202604062075	35.5 kB	323 ms
Helvetica-Regular.woff2	200	font	style.css?v=202604062075	109 kB	377 ms
detiknetwork.woff2	200	font	style.css?v=202604062075	0.5 kB	293 ms
Montserrat-Bold.woff	200	font	framebar.css?v=202604062075	60.3 kB	336 ms
JTUSjlg1_i6t8kCHKm459Wlhw.woff2	200	font	css2?family=Montserratwgb	38.0 kB	226 ms
Montserrat-SemiBold.woff	200	font	style.css?v=202604062075	60.2 kB	375 ms
channel-box-embed-4.html	200	document	(index):4449	1.8 kB	381 ms
bg-cb-kmp.webp	200	webp	style.css?v=202604062075	23.4 kB	207 ms
bg-cb-komdigi.webp	200	webp	style.css?v=202604062075	42.4 kB	208 ms